

iOS Seminar 3

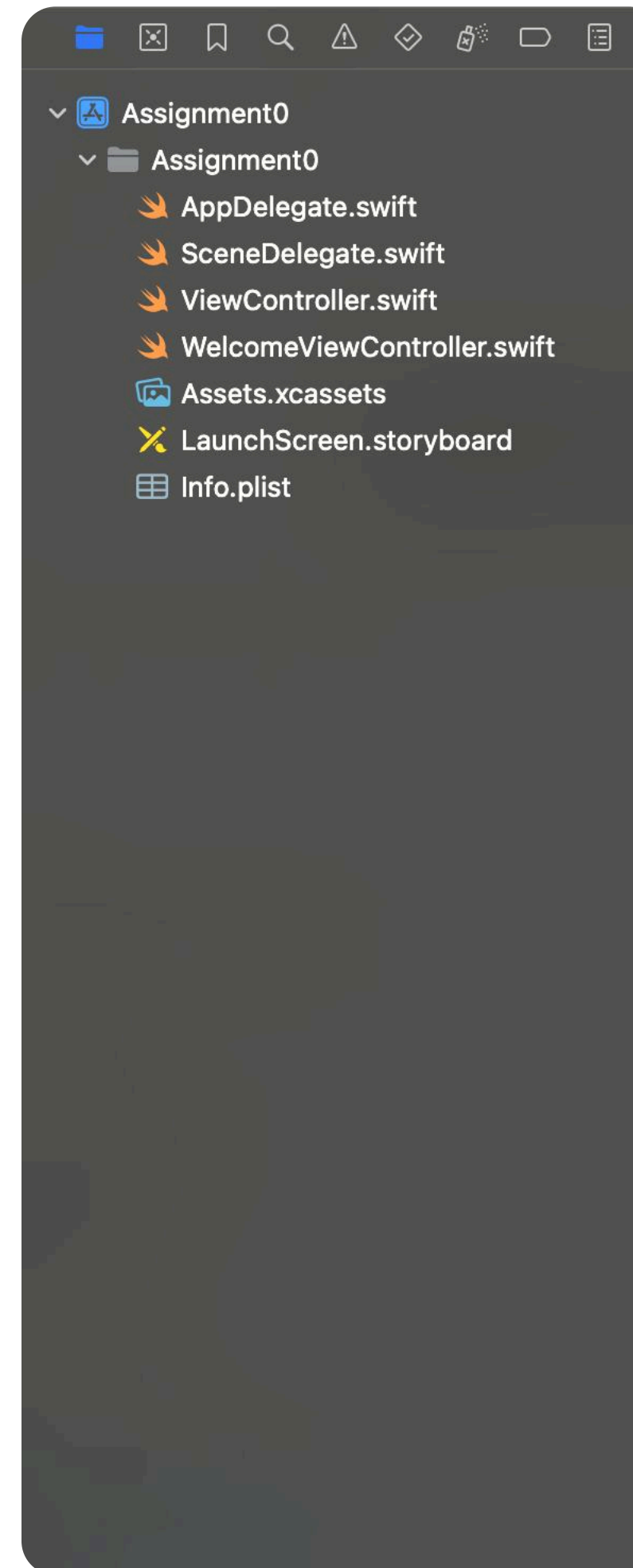
Wafflestudio 2025



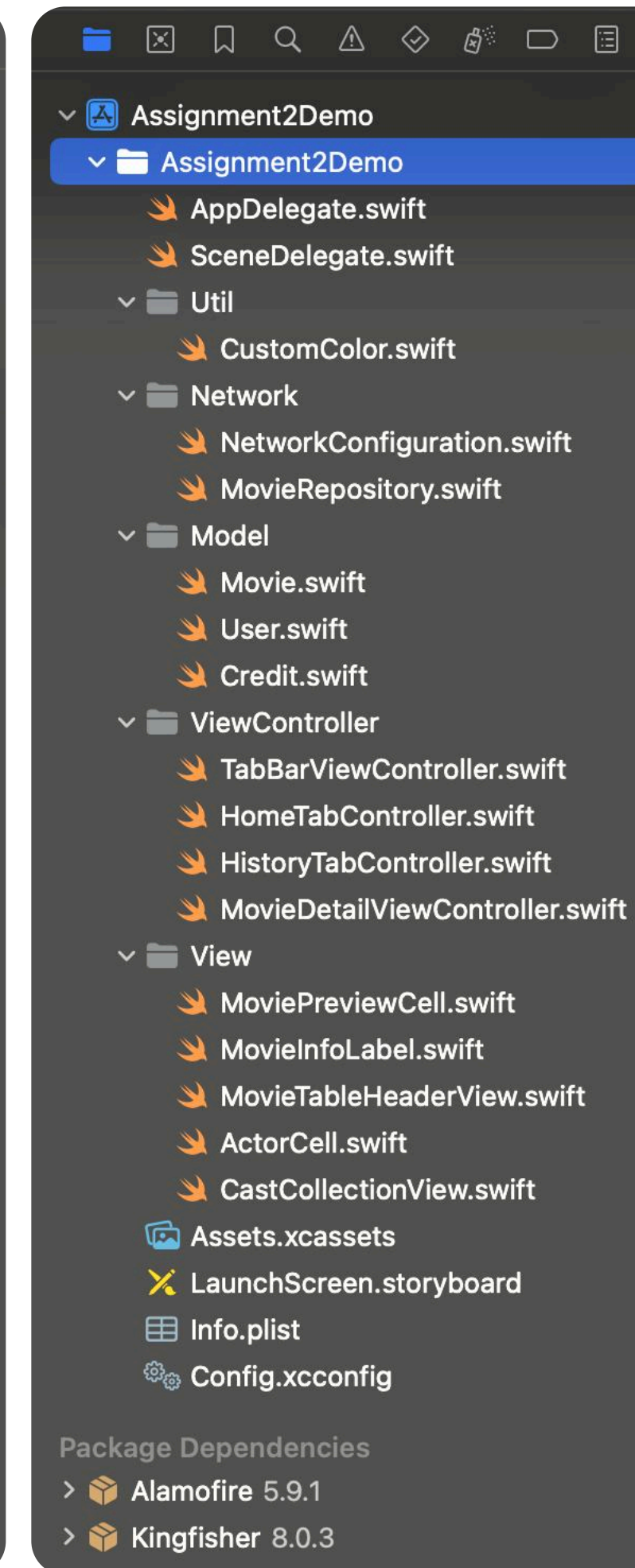
Design Pattern

Design Pattern

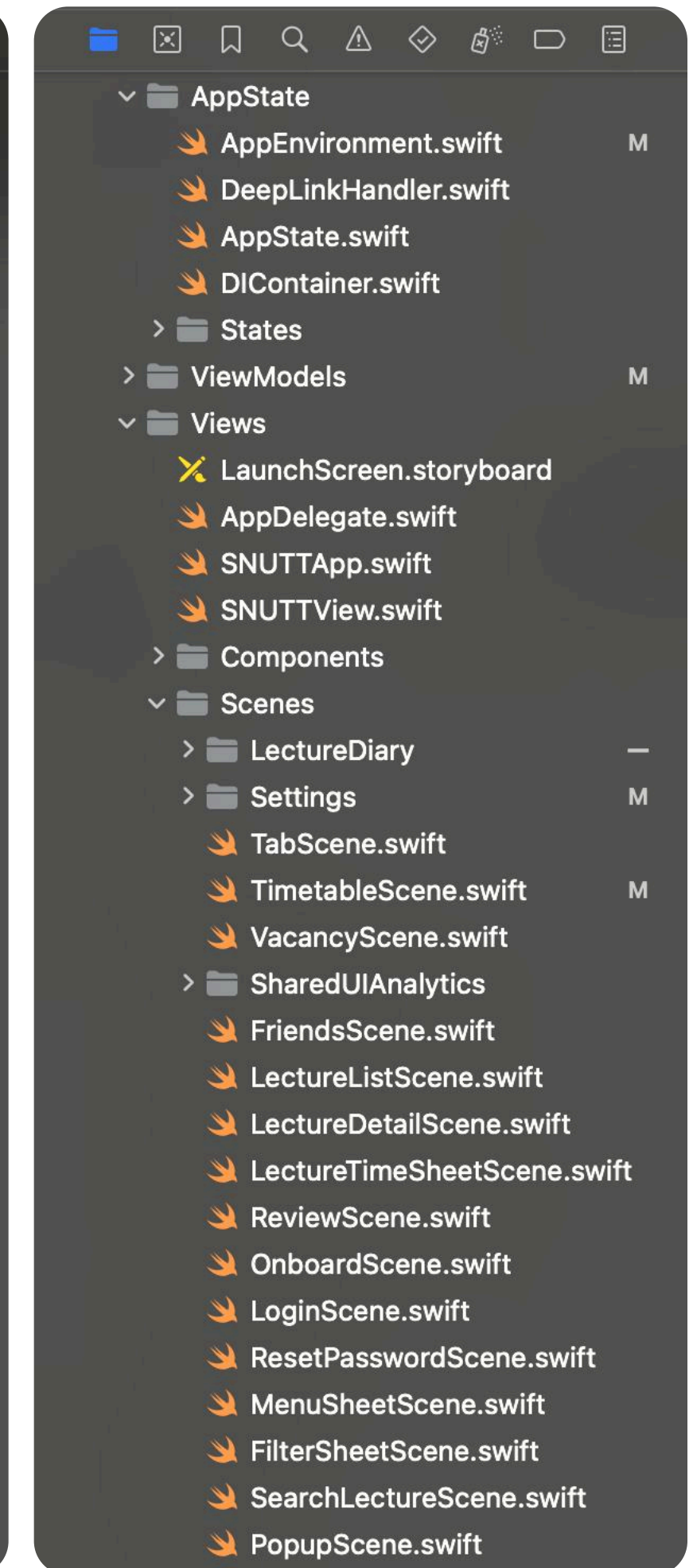
- 규모가 큰 프로젝트를 관리하기 위한 방법
 - 상태 관리, 네트워크 통신, 로컬 저장소
 - 수많은 UI 화면, 컴포넌트 관리
 - ex. MVC, MVVM, MVP 등...



Assignment0.xcodeproj



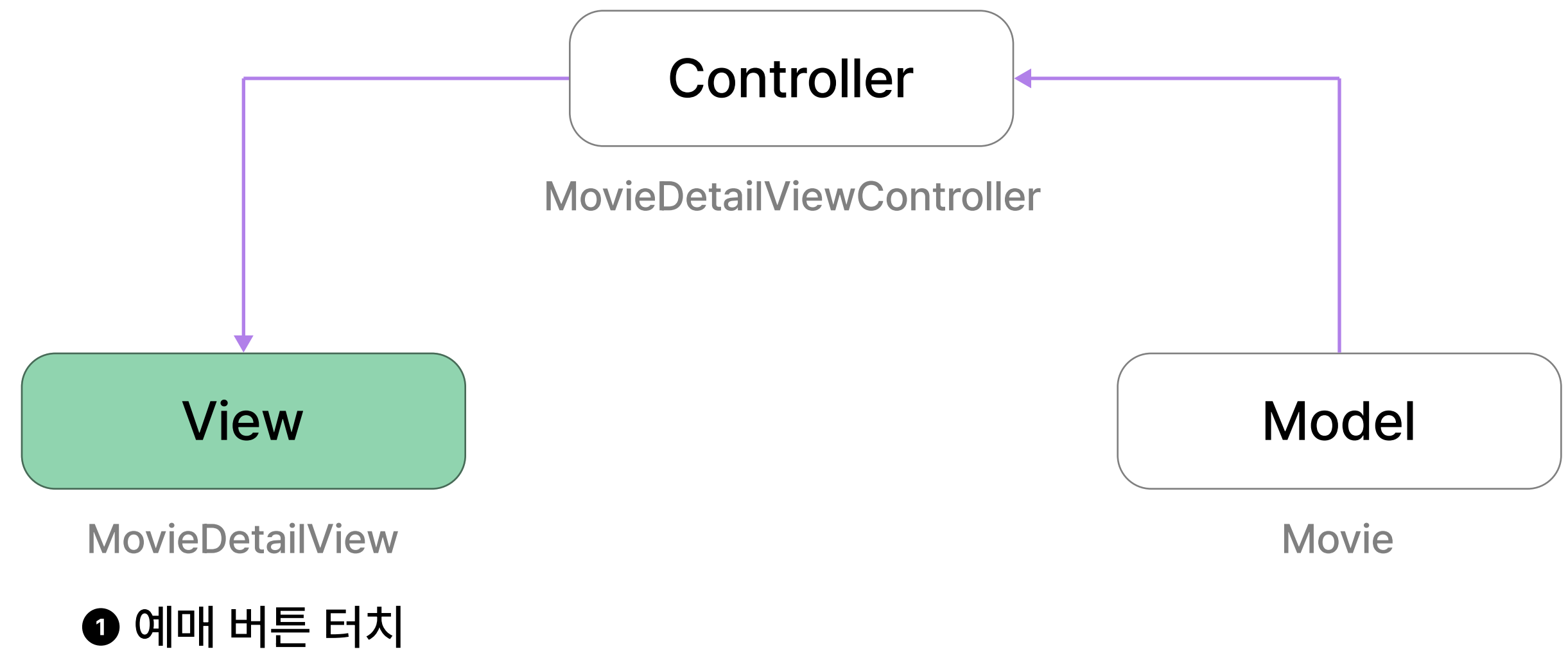
Assignment2Demo.xcodeproj



SNUTT.xcodeproj

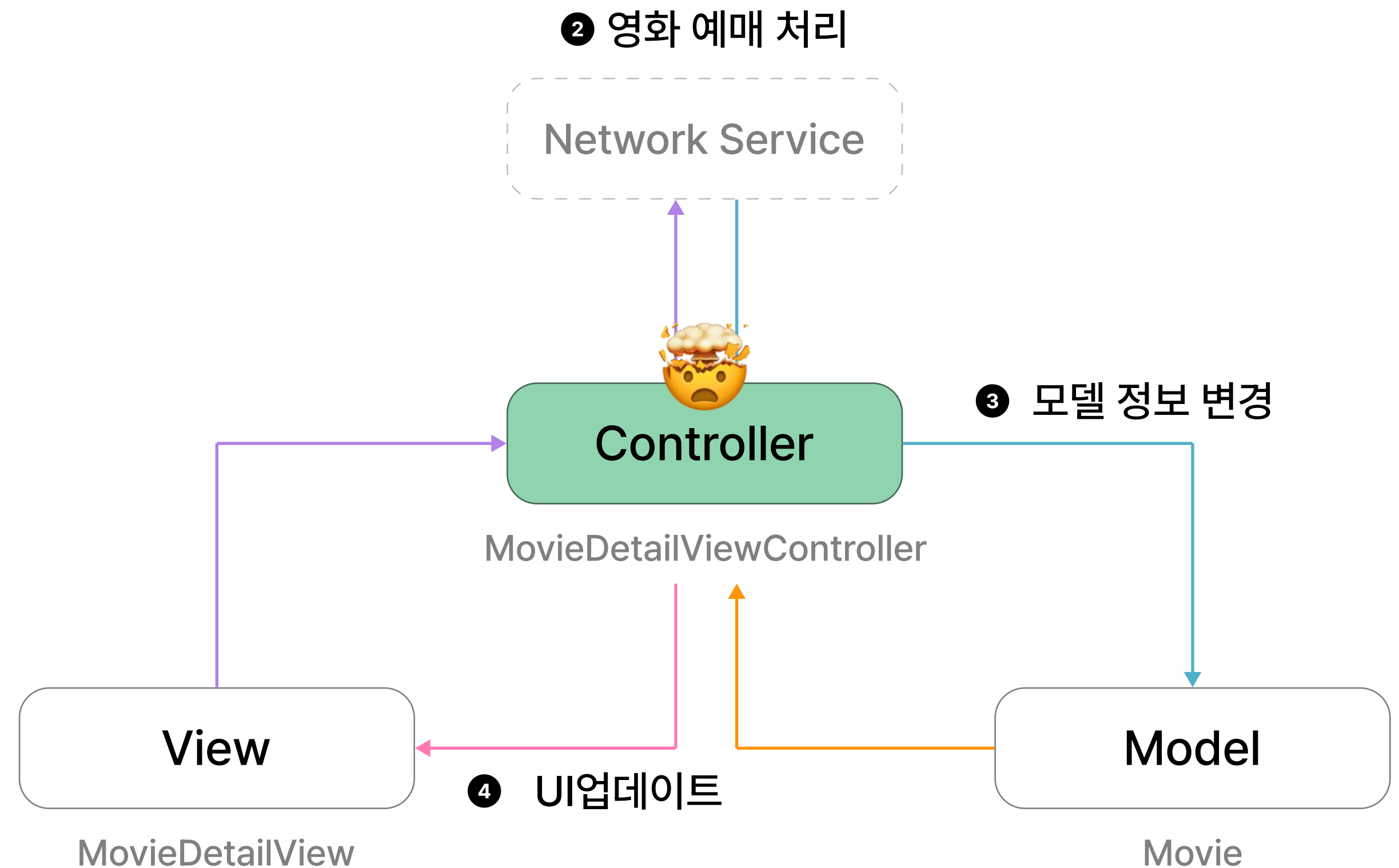
MVC

- Model - View - Controller
 - 이제까지 진행했던 과제와 유사한 패턴
 - UIKit 기본 패턴
- 일명 Massive - View - Controller
 - 하나의 ViewController가 책임지는 역할이 많다는 문제



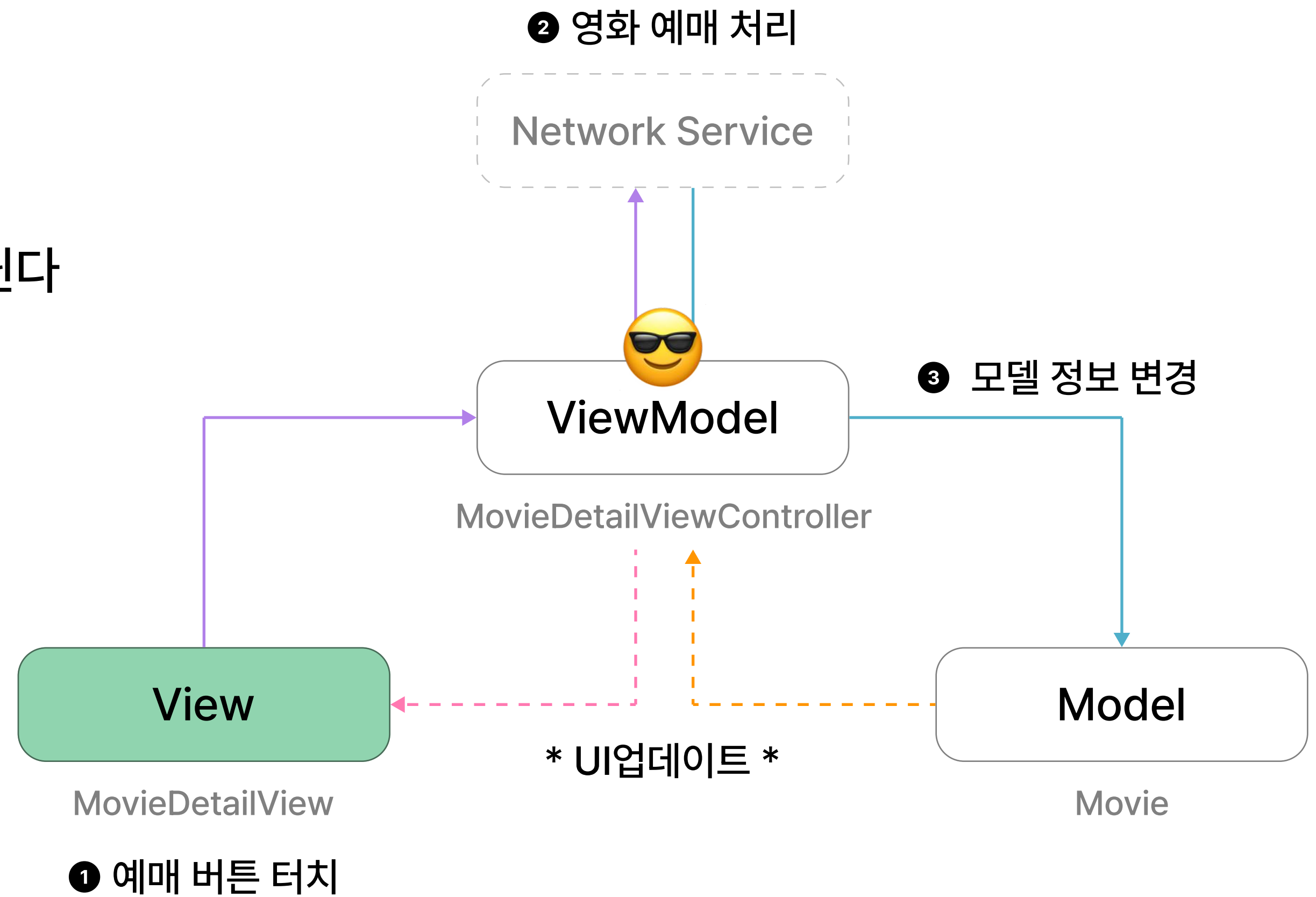
MVC

- Model - View - Controller
 - 이제까지 진행했던 과제와 유사한 패턴
 - UIKit 기본 패턴
- 일명 Massive - View - Controller
 - 하나의 ViewController가 책임지는 역할이 많다는 문제



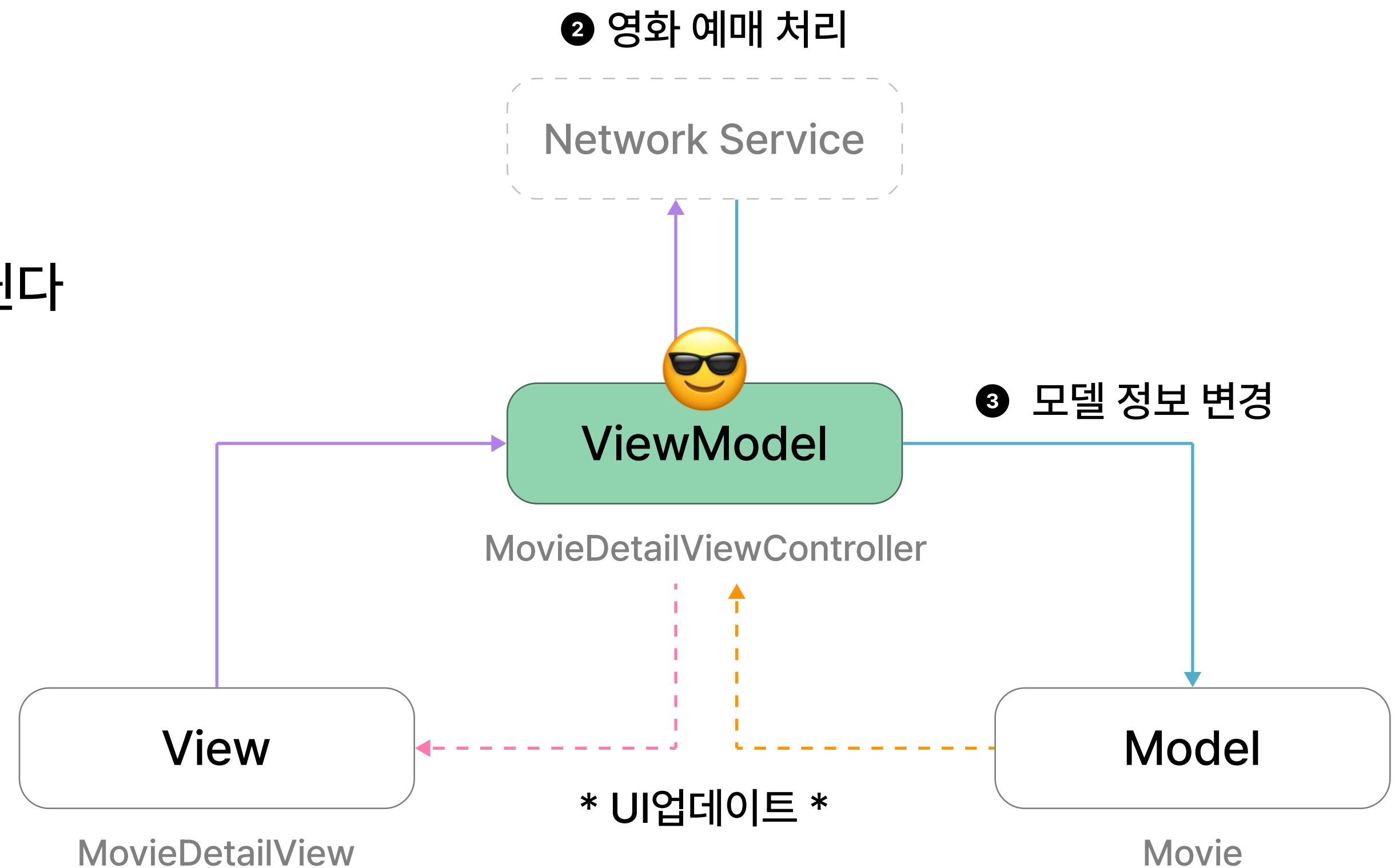
MVVM

- Model - View - ViewModel
- View와 Model 간의 결합도를 없앤 디자인 패턴
 - ViewModel이 바뀌면, View도 자동으로 바뀐다
 - 과제1에서 매번 `changeCellUI()` 등의 메소드를 호출했던 것과 비교



MVVM

- Model - View - ViewModel
- View와 Model 간의 결합도를 없앤 디자인 패턴
 - ViewModel이 바뀌면, View도 자동으로 바뀐다
 - 과제1에서 매번 `changeCellUI()` 등의 메소드를 호출했던 것과 비교



SwiftUI

SwiftUI

- WWDC 2019에서 발표한 프레임워크
- Declarative syntax
 - (UIKit은 Imperative)
- Data Driven
- HStack, VStack, ZStack, Spacer 등을 이용

```
8 import SwiftUI
9
10 struct MainView: View {
11
12     @State private var id: String = ""
13     @State private var password: String = ""
14     @State private var pushToAfterLoginView: Bool = false
15
16     var body: some View {
17         VStack {
18             VStack(alignment: .leading, spacing: 0) {
19                 Text("iOS Seminar")
20                     .font(.system(size: 28, weight: .semibold))
21
22                 Spacer().frame(height: 48)
23
24                 VStack(spacing: 32) {
25                     TextField("아이디", text: $id, prompt: Text("아이디(8자 이하)"))
26                         .font(.system(size: 17))
27                         .textInputAutocapitalization(.never)
28
29                     TextField("비밀번호", text: $password, prompt: Text("비밀번호"))
30                         .font(.system(size: 17))
31                         .textContentType(.password)
32                         .textInputAutocapitalization(.never)
33                 }
34             }
35             .padding(.horizontal, 24)
36
37             /// ...
```

SwiftUI로 구현한 과제0 MainView

UIKit보다 좋은 점?

- 과제1의 TodoTableViewCell을 SwiftUI로 구현한다면...
- ex) 좌우 간격 24, 상하 간격 16을 설정하고자 했을 때
 - UIKit

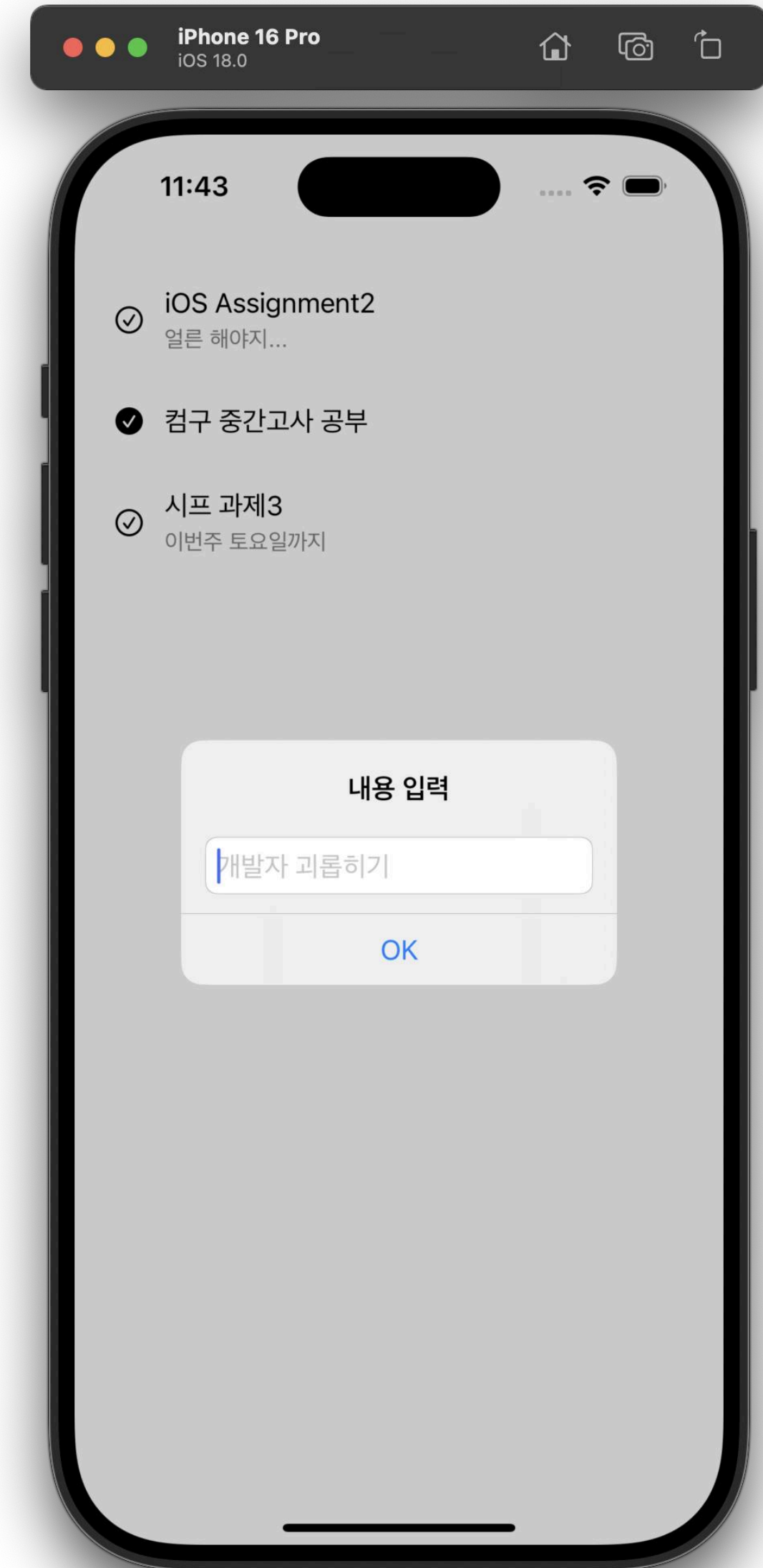
```
98     private func setLayoutConstraint() {  
99         addSubview(labelStackView)  
100         labelStackView.translatesAutoresizingMaskIntoConstraints = false  
101         NSLayoutConstraint.activate([  
102             labelStackView.leadingAnchor.constraint(equalTo: checkImageView.trailingAnchor, constant: 24),  
103             labelStackView.trailingAnchor.constraint(equalTo: safeAreaLayoutGuide.trailingAnchor, constant: -24),  
104             labelStackView.topAnchor.constraint(equalTo: safeAreaLayoutGuide.topAnchor, constant: 16),  
105             labelStackView.bottomAnchor.constraint(equalTo: safeAreaLayoutGuide.bottomAnchor, constant: -16),  
106         ])  
107     }
```

- SwiftUI

```
42         .padding(.horizontal, 24)  
43         .padding(.vertical, 16)
```

그럼에도 UIKit을 배우는 이유

- 복잡한/정교한 custom UI를 구현하고자 하는 경우
 - UIKit + SwiftUI 혼용하는 프로젝트도 다수 존재
- UIKit 전용 라이브러리를 사용해야 하는 경우
 - ex. Google Maps iOS SDK
- iOS 15까지 pure SwiftUI로는 Alert에 TextField를 넣을 수 없었다...



SwiftUI+Data Managing



SwiftUI가 View를 업데이트하는 방식 (공식 문서)

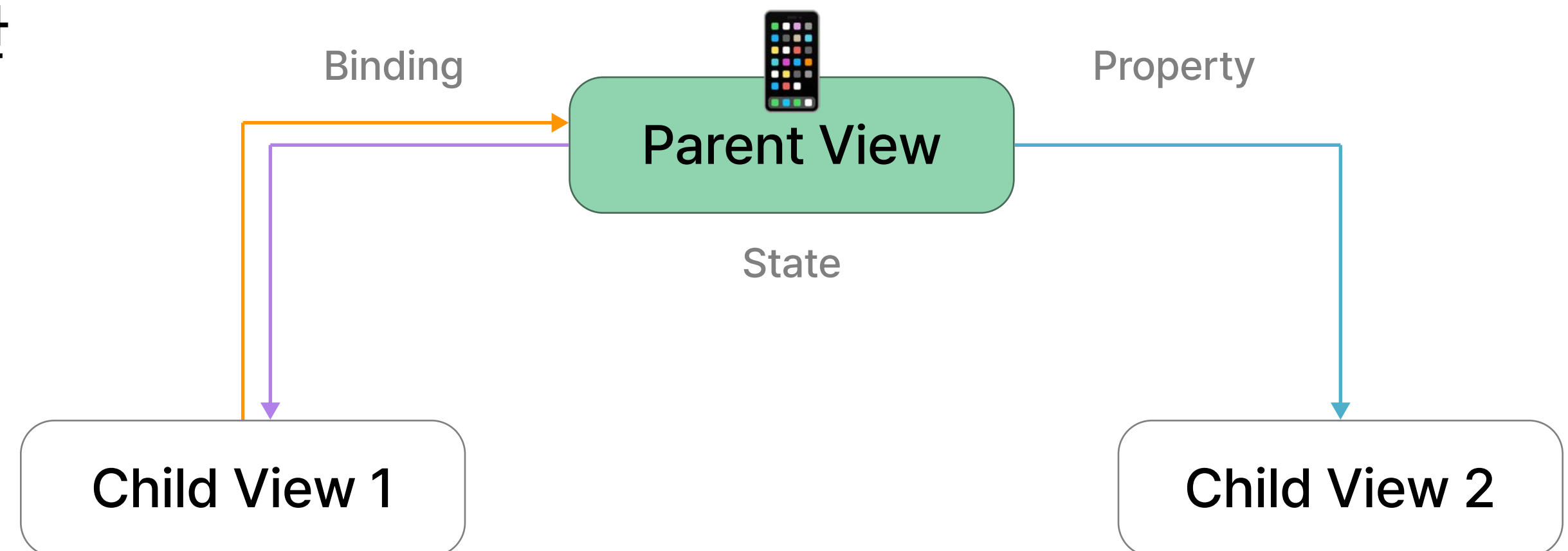
1. View State 공유
2. Model Data의 변경사항 관찰

View에 의존성이 있는 데이터가 업데이트되면, 그 데이터에 연관된 인터페이스만 자동으로 업데이트

→ 이 과정에서 `init()`이 여러 번 호출됨

View State Sharing (공식 문서)

- Store data as state in the least common ancestor of the views that need the data to establish a single source of truth that's shared across views.
- 다만 이러한 state는 persistent한 데이터보다는, User Interface에만 영향을 미치는 데이터에만 적용할 것을 권장
- 정리: SwiftUI에서는 View 업데이트를 위해 필요한 값을 State로 주고받는다!

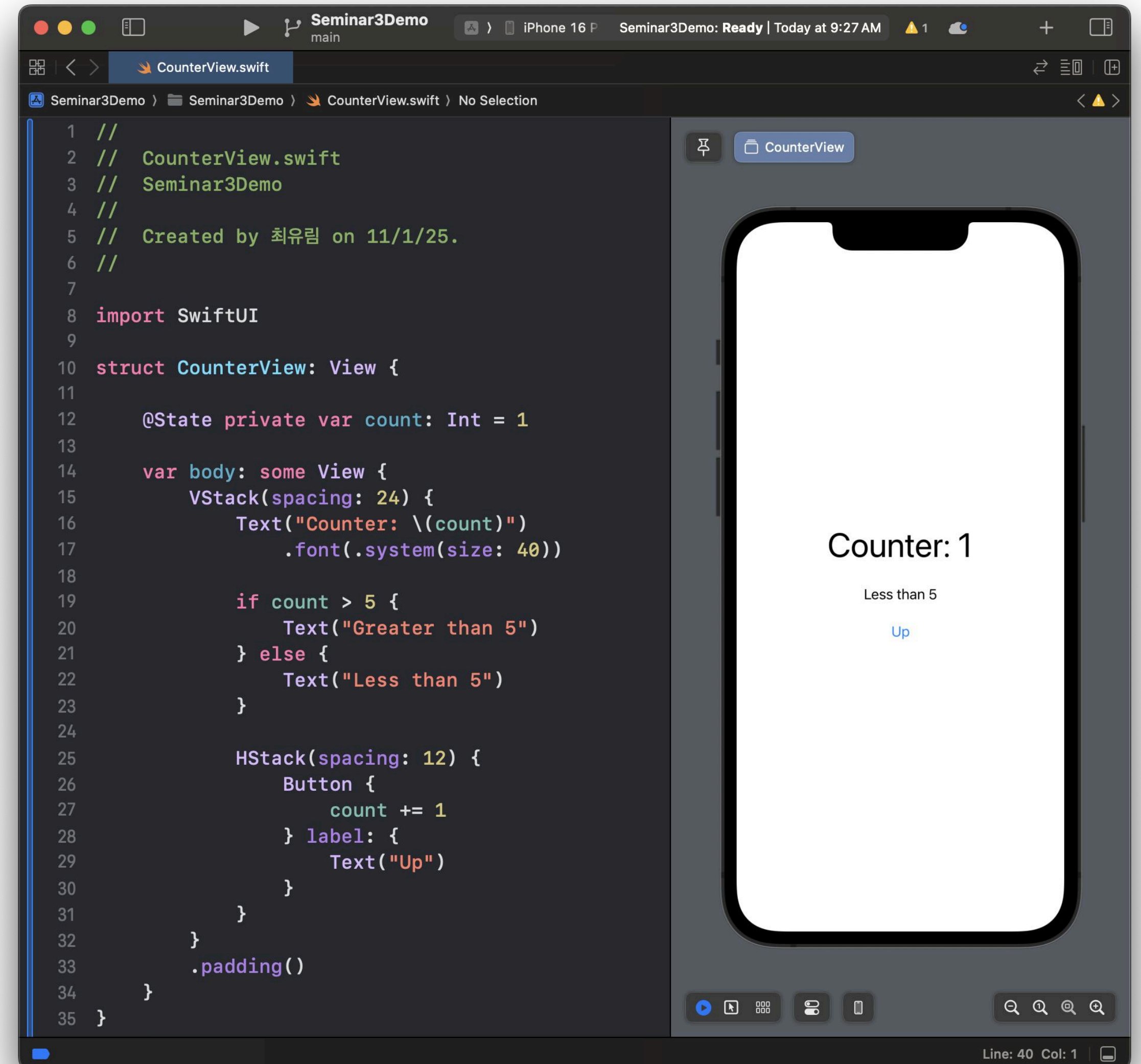


@State (공식 문서)

- User Interface state를 지역적으로 관리하는 경우
 - ex. 화면의 counter 값을 1씩 올리기
- 하위 View에게 값을 공유하는 방법
 - Read-only: 값 그대로 전달 (let, var)
 - Read-write: Binding 공유

@Binding (공식 문서)

- 상위 View ↔ 하위 View 데이터의 양방향 흐름
- ex. 과제1에서의 ColorMode Toggle



@State 예제

Seminar3Demo
main

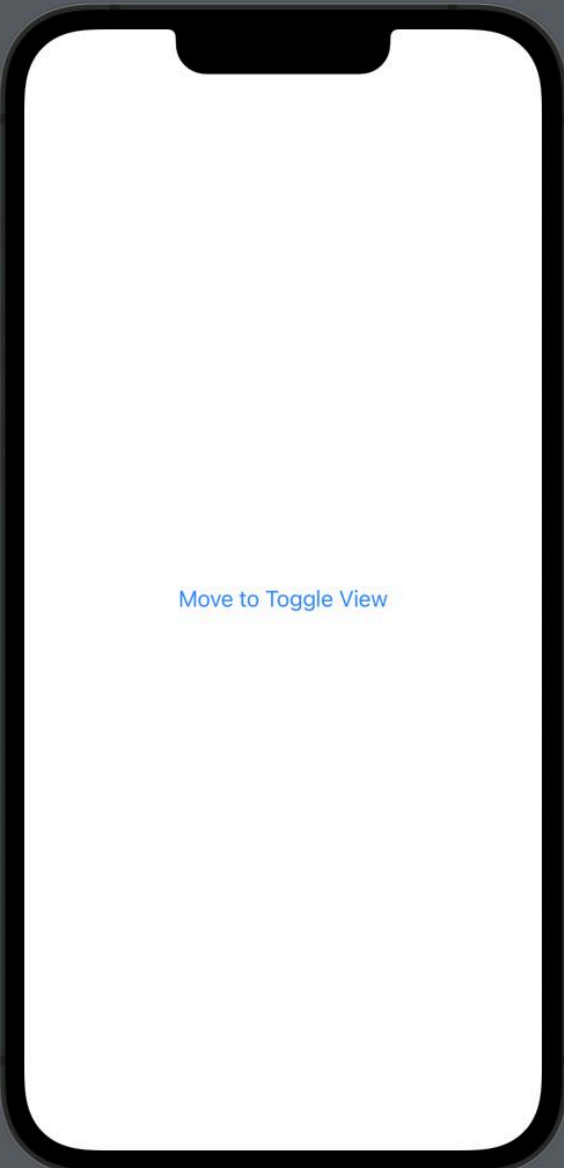
iPhone 16 Seminar3Demo: Ready | Today at 10:55 AM

CounterView.swift ToggleView.swift

Seminar3Demo > Seminar3Demo > ToggleView.swift > No Selection

```
1 //
2 // ToggleView.swift
3 // Seminar3Demo
4 //
5 // Created by 최유림 on 11/1/25.
6 //
7
8 import SwiftUI
9
10 struct MainView: View {
11
12     @State private var colorMode: Bool = false
13
14     var body: some View {
15         VStack {
16             NavigationLink {
17                 ToggleView(colorMode: $colorMode)
18             } label: {
19                 Text("Move to Toggle View")
20             }
21         }
22         .frame(maxWidth: .infinity, maxHeight: .infinity)
23         .background(colorMode ? .yellow : .white)
24     }
25 }
26
27
28
29
30
31
32
33
34
35
```

ToggleView



Line: 37 Col: 1

MainView.swift

Seminar3Demo
main

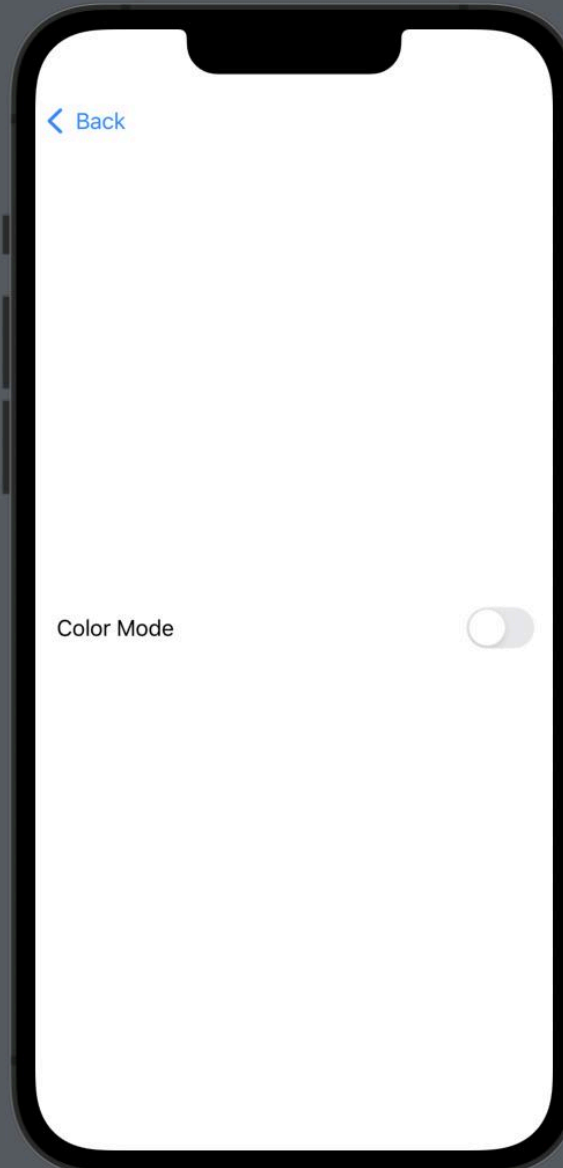
iPhone 16 Seminar3Demo: Ready | Today at 10:56 AM

CounterView.swift ToggleView.swift

Seminar3Demo > Seminar3Demo > ToggleView.swift > No Selection

```
1 //
2 // ToggleView.swift
3 // Seminar3Demo
4 //
5 // Created by 최유림 on 11/1/25.
6 //
7
8 import SwiftUI
9
10 struct ToggleView: View {
11
12     @Binding var colorMode: Bool
13
14     var body: some View {
15         VStack {
16             Toggle(isOn: $colorMode) {
17                 Text("Color Mode")
18             }
19             .tint(.black)
20         }
21         .padding()
22     }
23 }
24
25 #Preview {
26     NavigationView {
27         MainView()
28     }
29 }
30
31
32
33
34
35
```

ToggleView

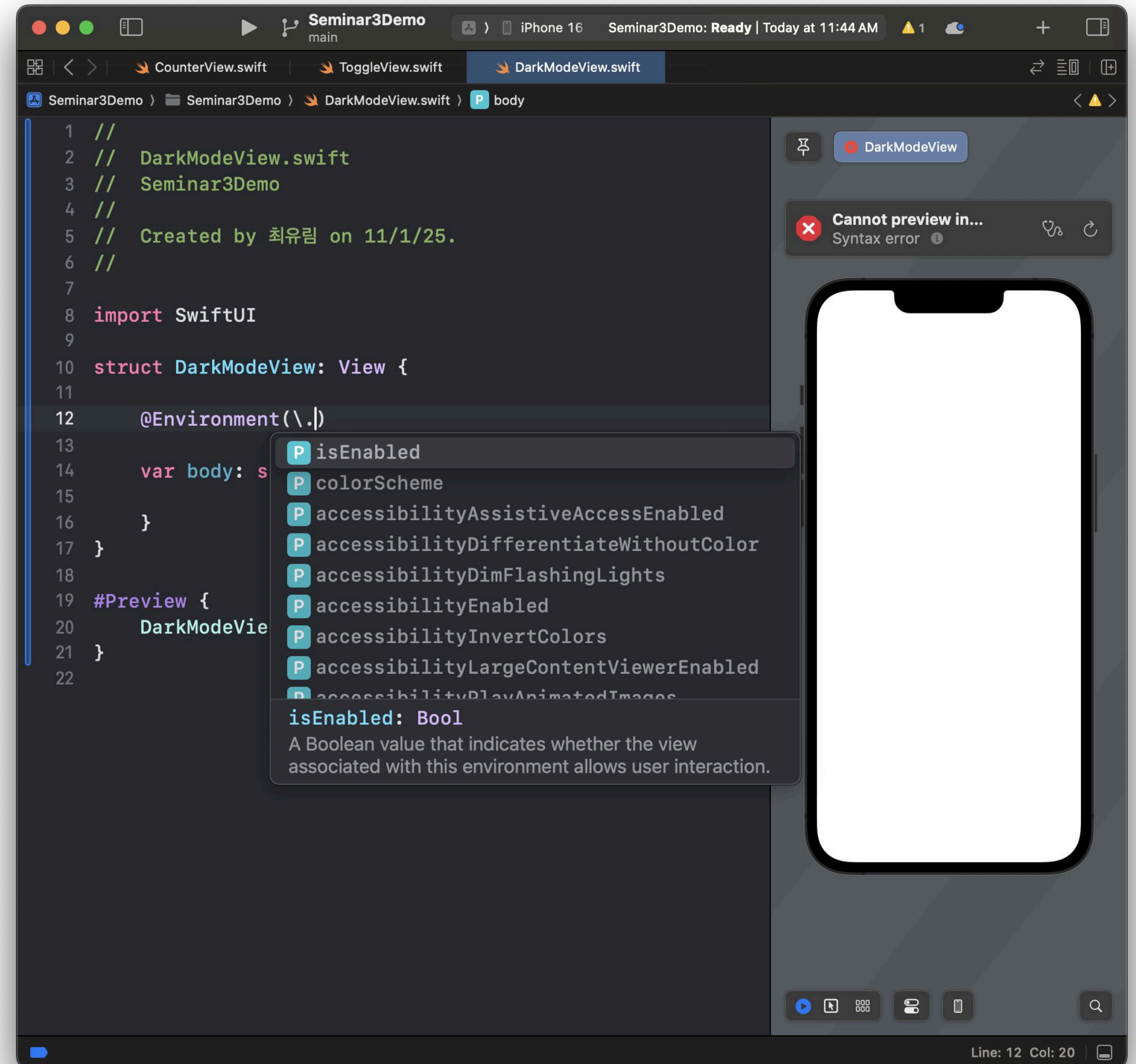


Line: 36 Col: 1

ToggleView.swift

@Environment (공식 문서)

- View의 Environment에서 제공하는 값 읽기 (Read-only)
 - ex. 핸드폰 디스플레이 다크모드 설정이 켜진 상태인지 확인하고 싶은 경우
- 기존의 EnvironmentValues를 override하거나, 커스텀 값/타입을 넘기는 것도 가능
- cf. @EnvironmentObject



어떤 방법이 제일 좋을까?

- 하위 View에서 값이 변경될 일이 없다 (Read-only)
→ Property (let)
- 하위 View에서 값을 변경해야 한다 (Read-write)
→ @Binding
- 상위 View ↔ 하위 View 계층이 많이 차이나는 경우
→ @Environment / @EnvironmentObject

SwiftUI Life cycle

SwiftUI Life cycle (공식 문서)

- UIKit을 사용하는 앱의 entry point: AppDelegate
- SwiftUI를 사용할 때 entry point: <MyAppName>App (App Protocol 채택)
- 공통점: @main annotation
 - 특정 structure, class, 혹은 enumeration이 top-level entry point임을 나타냄
 - C언어의 `int main(int argc, char *argv[])` 같은 느낌

```
8 import UIKit
9
10 @main
11 class AppDelegate: UIResponder, UIApplicationDelegate {
12
13     func application(_ application: UIApplication, didFinishLaunchingWithOptions
14         [UIApplication.LaunchOptionsKey: Any]?) -> Bool {
15         // Override point for customization after application launch.
16         return true
17     }
18 }
```

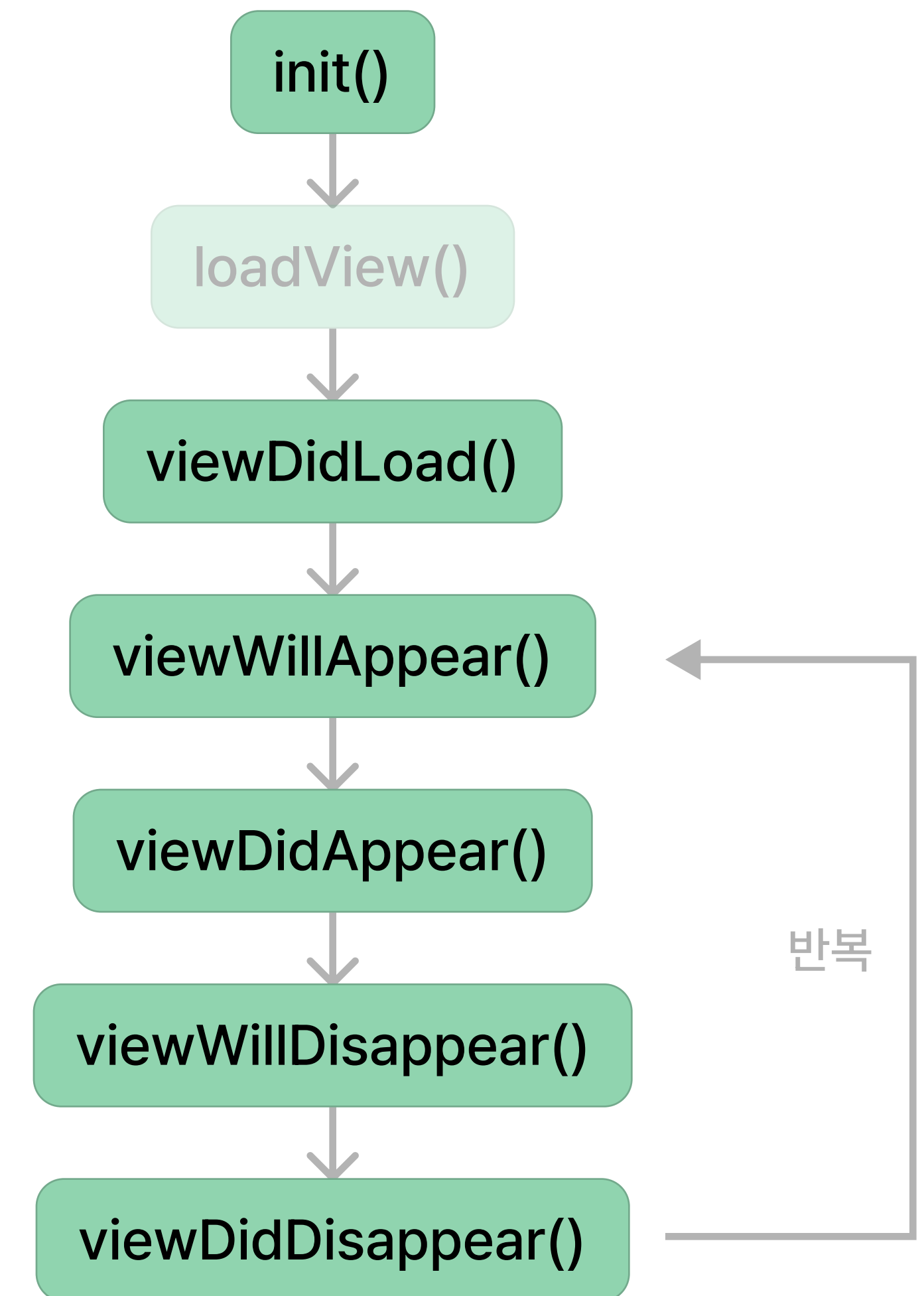
UIKit 예시

```
8 import SwiftUI
9
10 @main
11 struct Seminar3DemoApp: App {
12     var body: some Scene {
13         WindowGroup {
14             ContentView()
15         }
16     }
17 }
```

SwiftUI 예시

SwiftUI View Life cycle

- 기본적으로 SwiftUI의 View는 Struct 기반 ($\leftrightarrow$ UIKit: Class 기반)
 - `init()`은 여러 번 호출된다
 - View를 재사용하기보다, 필요할 때마다 계속해서 새로 생성
- `onAppear`
 - (공식문서) A view that triggers action before it appears
 - (UIKit) `viewWillAppear`
- `onDisappear`
 - (공식문서) A view that triggers action after it disappears
 - (UIKit) `viewDidDisappear`



UIKit ViewController Life Cycle

```
8 import SwiftUI
9
10 struct FirstView: View {
11     var body: some View {
12         NavigationView {
13             VStack(spacing: 24) {
14                 Text("First View")
15                     .font(.system(size: 20))
16                 NavigationLink {
17                     SecondView()
18                 } label: {
19                     Text("Visit Second View")
20                 }
21             }
22             .padding()
23             .onAppear {
24                 print("First View onAppear")
25             }
26             .onDisappear {
27                 print("First View onDisappear")
28             }
29         }
30     }
31 }
```

FirstView.swift

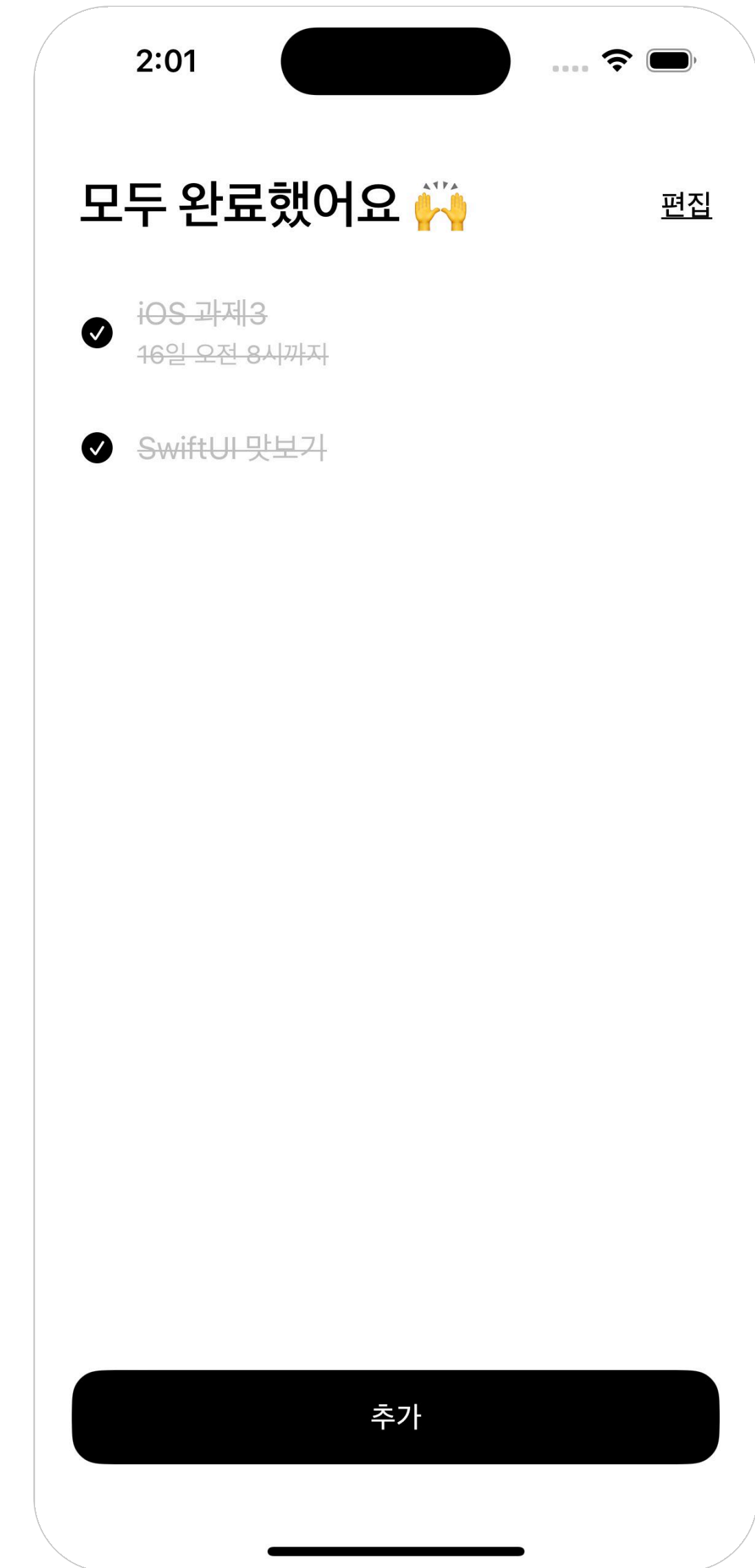
```
8 import SwiftUI
9
10 struct SecondView: View {
11     var body: some View {
12         VStack {
13             Text("Second View")
14         }
15         .padding()
16         .onAppear {
17             print("Second View onAppear")
18         }
19         .onDisappear {
20             print("Second View onDisappear")
21         }
22     }
23 }
```

SecondView.swift

과제 3 안내

SwiftUI를 이용한 Todo List 앱 만들기

- 과제 3 디자인 링크(Figma)
- 과제 목표
 - SwiftUI를 이용한 state 관리 및 UI 구현
- 관련 키워드
 - @State, @Binding, @Environment
 - NavigationView / NavigationLink
 - UserDefaults

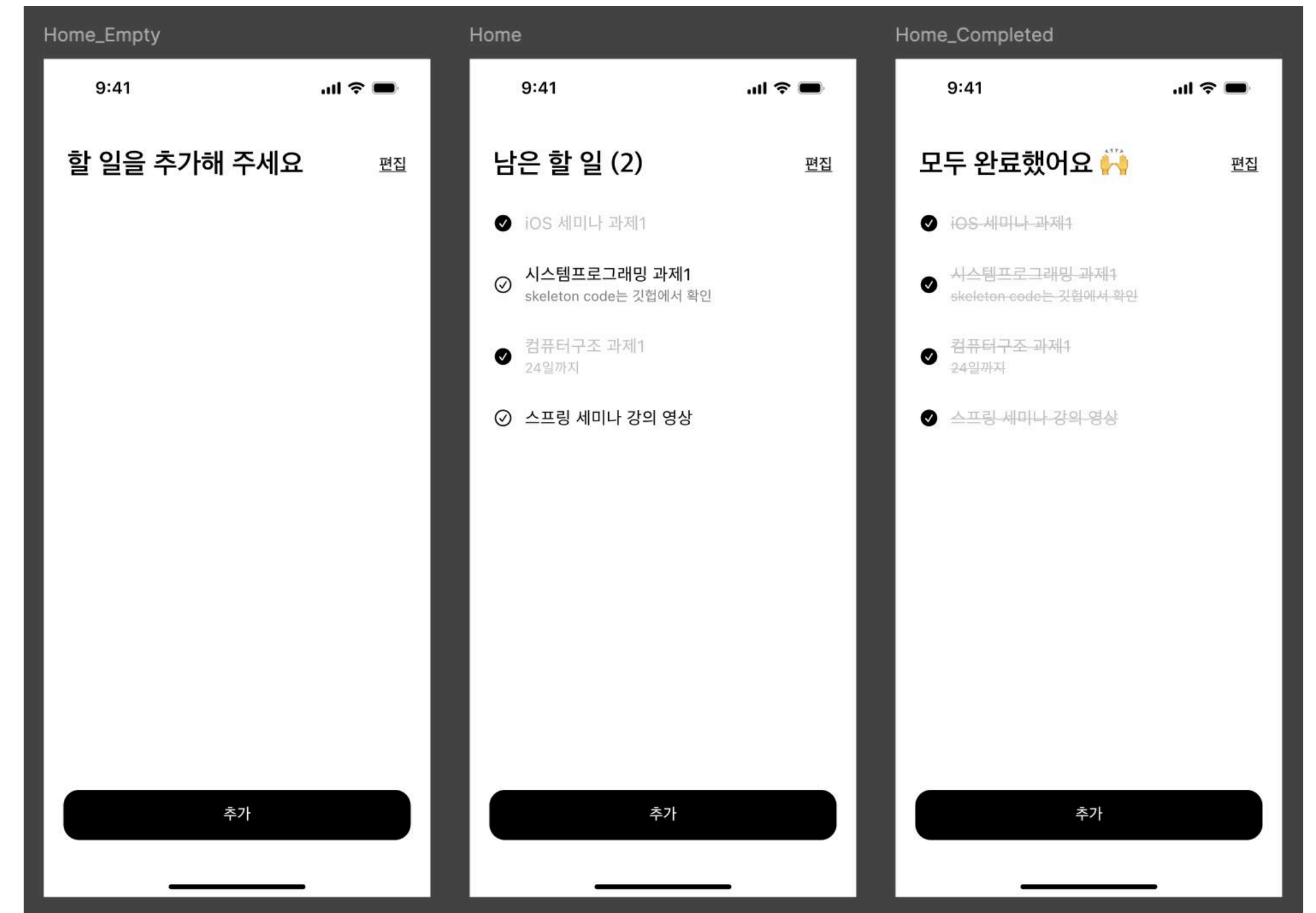


과제 상세 스펙 및 주의사항 (1)

- 전체
 - UI 구현은 SwiftUI만 사용합니다
 - 아직 강의 자료 15페이지에서의 “Model data”의 변경사항 관찰에 대한 내용을 다루지 않아 완전한 MVVM 구조를 짜기 어렵습니다.
미리 관련한 내용을 공부하고 MVVM 구조로 짜도 좋습니다
(관련 키워드 : @ObservableObject @ObservedObject @StateObject @Published)
 - Text, Color, Constraint, Corner Radius 등은 피그마에 주어진 값을 사용합니다
- Alert
 - 별도의 커스텀 디자인이 필요하지 않습니다. 스펙에 맞는 문구만 지정해 주세요

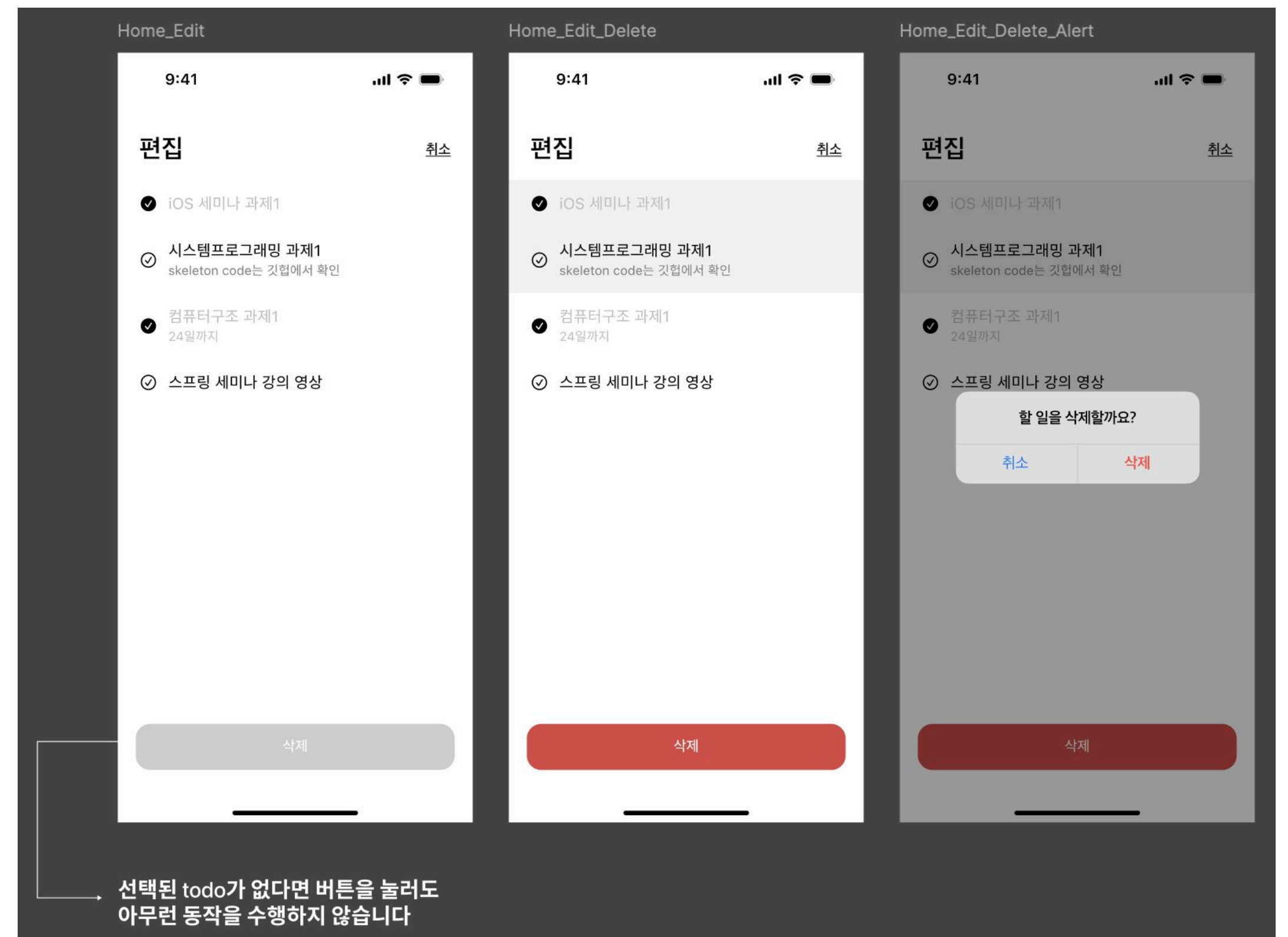
과제 상세 스펙 및 주의사항 (2)

- 홈 화면 (기본 모드)
 - 앱에 처음 진입하는 경우에는 언제나 기본 모드를 보여줍니다
 - 추가/삭제된 Todo의 목록과 각 Todo의 완료 여부는 앱을 종료 후 다시 실행하더라도 유지됩니다
 - Todo 목록의 상태에 따라 상단 문구를 알맞게 보여줍니다
 - NavigationBar Title이 아닌, Custom View입니다
 - 체크 아이콘을 탭하면 Todo의 완료 여부를 전환합니다
 - 구현에 따라 Cell 영역 어디든 탭해도 Todo의 완료 여부가 전환되도록 해도 괜찮습니다
 - 완료된 Todo는 취소선을 보여줍니다. 달라지는 Text 색상에 유의합니다



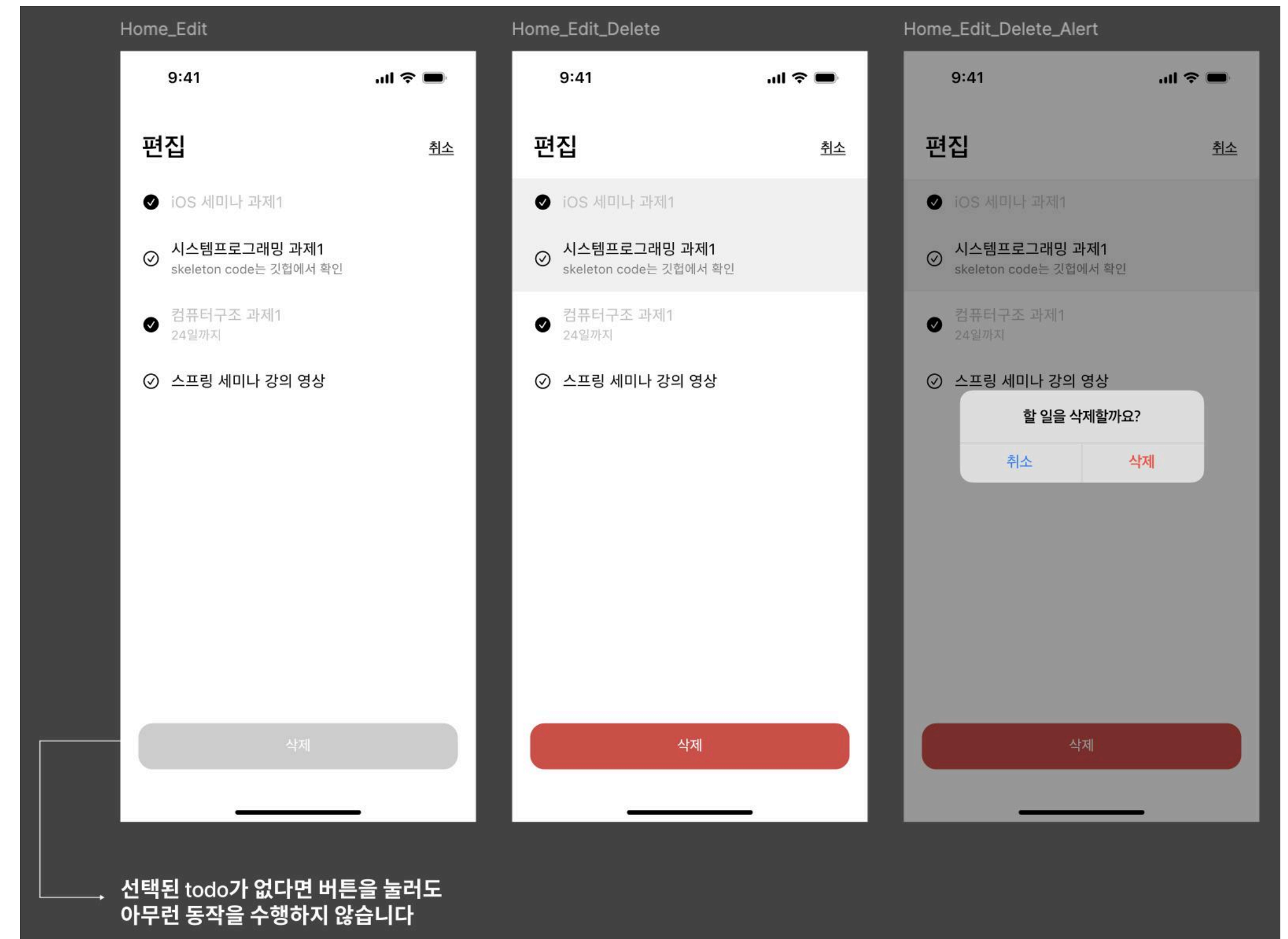
과제 상세 스펙 및 주의사항 (3)

- 홈 화면 (편집 모드)
 - 기본 모드에서 “편집” 버튼을 누르면 편집 모드로 전환됩니다
 - Cell을 탭하면 배경 색상이 바뀌며 Cell이 선택됩니다
 - 이미 선택된 Cell을 다시 탭하면 배경이 투명해지며 미선택 상태로 돌아갑니다
- 삭제 버튼
 - 선택된 Cell이 없는 경우에는 삭제 버튼을 탭해도 아무런 동작을 수행하지 않습니다
 - 선택된 Cell이 하나 이상인 경우에는 Alert를 띄웁니다
 - 두 상태에 따라 다른 배경 색상에 유의합니다
- 취소 버튼을 누르면 선택 내역을 초기화하고 기본 모드로 돌아갑니다



과제 상세 스펙 및 주의사항 (4)

- Alert
 - “삭제” 누르면 선택했던 Todo를 삭제합니다
 - “취소”를 누르면 편집 화면으로 되돌아갑니다
 - 선택했던 Todo 목록이 초기화되지 않습니다



과제 상세 스펙 및 주의사항 (5)

- Todo 추가 화면
 - 저장하지 않고 화면을 나가는 경우에는 작성 중이던 Todo가 초기화됩니다
 - 입력 중에는 X 버튼이 뜨도록 합니다
 - 자동으로 첫 글자를 대문자로 만드는 기능을 비활성화합니다
- Alert
 - “저장” 버튼을 눌렀을 때, Todo의 제목이 비어있는 경우에는 Alert를 띄웁니다
 - Todo의 제목이 비어있지 않다면, 새 Todo를 만들고 이전 화면으로 되돌아갑니다
 - 홈 화면에는 새로 추가된 Todo가 바로 보여야 합니다

